

```

1 // Comments marked by initials ER, KP, KL, DS, EC
2 //#####
3 // FILE:   FinalProjectStarter_main.c
4 //
5 // TITLE:   Final Project Starter
6 //#####
7
8 // Included Files
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15 #include "F28x_Project.h"
16 #include "F2837xD_SWPrioritizedIsrLevels.h"
17 #include "driverlib.h"
18 #include "device.h"
19 #include "F28379dSerial.h"
20 #include "song.h"
21 #include "dsp.h"
22 #include "fpu32/fpu_rfft.h"
23 #include "xy.h"
24 #include "MatrixMath.h"
25 #include "SE423Lib.h"
26 #include "OptiTrack.h"
27
28 #define PI          3.1415926535897932384626433832795
29 #define TWOPI       6.283185307179586476925286766559
30 #define HALFPi      1.5707963267948966192313216916398
31 // The Launchpad's CPU Frequency set to 200 you should not change this value
32 #define LAUNCHPAD_CPU_FREQUENCY 200
33
34 // ----- code for CAN start here -----
35 #include "F28379dCAN.h"
36 // #define TX_MSG_DATA_LENGTH      4
37 // #define TX_MSG_OBJ_ID          0 //transmit
38
39 #define RX_MSG_DATA_LENGTH      8
40 #define RX_MSG_OBJ_ID_1        1 //measurement from sensor 1
41 #define RX_MSG_OBJ_ID_2        2 //measurement from sensor 2
42 #define RX_MSG_OBJ_ID_3        3 //quality from sensor 1
43 #define RX_MSG_OBJ_ID_4        4 //quality from sensor 2
44 // ----- code for CAN end here -----
45
46 // Interrupt Service Routines predefinition
47 __interrupt void cpu_timer0_isr(void);
48 __interrupt void cpu_timer1_isr(void);
49 __interrupt void cpu_timer2_isr(void);
50 __interrupt void SWI1_HighestPriority(void);
51 __interrupt void SWI2_MiddlePriority(void);
52 __interrupt void SWI3_LowestPriority(void);
53 __interrupt void ADCC_ISR(void);
54 __interrupt void SPIB_isr(void);
55 // ----- code for CAN start here -----
56 __interrupt void can_isr(void);
57 // ----- code for CAN end here -----
58
59 void setF28027EPWM1A(float controleffort);
60 int16_t EPwm1A_F28027 = 1500;
61 void setF28027EPWM2A(float controleffort);
62 int16_t EPwm2A_F28027 = 1500;
63
64
65 // ----- code for CAN start here -----
66 // volatile uint32_t txMsgCount = 0;
67 // extern uint16_t txMsgData[4];
68
69 volatile uint32_t rxMsgCount_1 = 0;
70 volatile uint32_t rxMsgCount_2 = 0;
71 extern uint16_t rxMsgData[8];
72
73 uint32_t dis_raw_1[2];
74 uint32_t dis_raw_2[2];
75 uint32_t dis_1 = 0;
76 uint32_t dis_2 = 0;
77
78 uint32_t quality_raw_1[4];
79 uint32_t quality_raw_2[4];
80 float quality_1 = 0.0;
81 float quality_2 = 0.0;
82
83 uint32_t lightlevel_raw_1[4];
84 uint32_t lightlevel_raw_2[4];
85 float lightlevel_1 = 0.0;
86 float lightlevel_2 = 0.0;
87
88 uint32_t measure_status_1 = 0;
89 uint32_t measure_status_2 = 0;
90
91 volatile uint32_t errorFlag = 0;
92 // ----- code for CAN end here -----
93
94 uint32_t numTimer0calls = 0;
95 uint16_t UARTPrint = 0;
96
97 float printLV1 = 0;
98 float printLV2 = 0;
99
100 float printLinux1 = 0;
101 float printLinux2 = 0;
102
103 uint16_t LADARi = 0;
104 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
105 uint16_t G_len = 11; //length of command
106 xy ladar_pts[228]; //xy data
107 float LADARrightfront = 0;
108 float LADARfront = 0;
109 float LADARleftfront = 0;
110
111 float LADARtemp_x = 0;
112 float LADARtemp_y = 0;
113 extern datapts ladar_data[228];
114
115 extern uint16_t newLinuxCommands;
116 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
117
118 extern uint16_t NewLVData;
119 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
120 extern LVSendFloats_t DataToLabView;
121 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
122 extern uint16_t LADARpingpong;
123 extern uint16_t NewCAMdataThreshold1; // Flag new data
124 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
125 extern uint16_t NewCAMdataThreshold2; // Flag new data
126 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
127 extern uint16_t NewCAMdataThreshold3; // Flag new data
128 extern float fromCAMvaluesThreshold3[CAMNUM_FROM_FLOATS];
129
130 extern uint16_t NewCAMdataAprilTag1; // Flag new data
131 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
132
133 float tagid = 0;
134 float tagx = 0;
135 float tagy = 0;
136 float tagz = 0;
137 float tagthetax = 0;
138 float tagthetay = 0;
139 float tagthetaz = 0;
140 int32_t numtag = 0;

```

```

141 int32_t AprilTagState = 10;
142 float KpAprilTag = -1.0;
143 float AprilTagSetPoint = 0;
144 int32_t AprilTagCount = 0;
145
146 float MaxAreaThreshold1 = 0;
147 float MaxColThreshold1 = 0;
148 float MaxRowThreshold1 = 0;
149 float NextLargestAreaThreshold1 = 0;
150 float NextLargestColThreshold1 = 0;
151 float NextLargestRowThreshold1 = 0;
152 float NextNextLargestAreaThreshold1 = 0;
153 float NextNextLargestColThreshold1 = 0;
154 float NextNextLargestRowThreshold1 = 0;
155
156 float MaxAreaThreshold2 = 0;
157 float MaxColThreshold2 = 0;
158 float MaxRowThreshold2 = 0;
159 float NextLargestAreaThreshold2 = 0;
160 float NextLargestColThreshold2 = 0;
161 float NextLargestRowThreshold2 = 0;
162 float NextNextLargestAreaThreshold2 = 0;
163 float NextNextLargestColThreshold2 = 0;
164 float NextNextLargestRowThreshold2 = 0;
165
166 //EC
167 float MaxAreaThreshold3 = 0;
168 float MaxColThreshold3 = 0;
169 float MaxRowThreshold3 = 0;
170 float NextLargestAreaThreshold3 = 0;
171 float NextLargestColThreshold3 = 0;
172 float NextLargestRowThreshold3 = 0;
173 float NextNextLargestAreaThreshold3 = 0;
174 float NextNextLargestColThreshold3 = 0;
175 float NextNextLargestRowThreshold3 = 0;
176
177 uint32_t numThres1 = 0;
178 uint32_t numThres2 = 0;
179 uint32_t numThres3 = 0; //EC
180
181 pose ROBOTps = {0,0,0}; //robot position
182 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
183 float LADARxoffset = 0;
184 float LADARyoffset = 0;
185
186 uint32_t timecount = 0;
187 int16_t RobotState = 1;
188 int32_t checkfrontally = 0;
189 int32_t WallFollowTime = 0;
190
191 #define NUMWAYPOINTS 10 //Modified NUMWAYPOINTS from 8 to 10 for two extra waypoint targets for the robot path - DS
192 uint16_t statePos = 0;
193 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
194 uint16_t i = 0;//for loop
195
196 uint16_t right_wall_follow_state = 2; // right follow
197 float Kp_front_wall = -2.0;
198 float front_turn_velocity = 0.2;
199 float left_turn_Stop_threshold = 3.5;
200 float Kp_right_wal = -4.0;
201 float ref_right_wall = 1.1;
202 float foward_velocity = 1.0;
203 float left_turn_Start_threshold = 1.3;
204 float turn_saturation = 2.5;
205
206 uint16_t left_wall_follow_state = 2; //DS: left wall follow state
207 float Kp_left_wal = 4.0; //DS: opposite sign of right wall follow
208 float ref_left_wall = 1.1; //DS: same desired wall distance as right wall
209 float right_turn_Stop_threshold = 3.5;
210 float right_turn_Start_threshold = 1.3;
211
212 //RC Servo
213 float Gate_0 = -60 ;
214 float Gate_C = 0 ;
215 float RTong = -49.34; //KLEC: ball goes to the left
216 float MTong = -11.49;
217 float LTong = 1.47; //KLEC: ball goes to the right
218 float dropOffStart = 0;
219
220 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
221
222 //more kalman vars
223 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
224 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
225 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
226 float z[3][1]; // state measurement
227 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
228 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
229 #define ProcUncert 0.0001
230 #define CovScalar 10
231 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
232 {0,ProcUncert,ProcUncert/CovScalar},
233 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
234 #define MeasUncert 1
235 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
236 {0,MeasUncert,MeasUncert/CovScalar},
237 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
238 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
239 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
240 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
241 float temp_3x3[3][3]; // intermediate storage
242 float temp_3x1[3][1]; // intermediate storage
243 float ytilde[3][1]; // difference between predictions
244
245 // Optitrack Variables
246 int32_t OptiNumUpdatesProcessed = 0;
247 pose OPTITRACKps;
248 extern uint16_t new_optitrack;
249 extern float Optitrackdata[OPTITRACKDATASIZE];
250 int16_t newOPTITRACKpose=0;
251
252 int16_t adcc2result = 0;
253 int16_t adcc3result = 0;
254 int16_t adcc4result = 0;
255 int16_t adcc5result = 0;
256 float adcc2Volt = 0.0;
257 float adcc3Volt = 0.0;
258 float adcc4Volt = 0.0;
259 float adcc5Volt = 0.0;
260 int32_t numADCCalls = 0;
261 float LeftWheel = 0;
262 float RightWheel = 0;
263 float LeftWheel_1 = 0;
264 float RightWheel_1 = 0;
265 float LeftVel = 0;
266 float RightVel = 0;
267 float uLeft = 0;
268 float uRight = 0;
269 float HandValue = 0;
270 float vref = 0;
271 float turn = 0;
272 float gyro9250_drift = 0;
273 float gyro9250_angle = 0;
274 float old_gyro9250 = 0;
275 float gyrolPRS10_angle = 0;
276 float gyrolPRS10_offset = 0;
277 float gyrolPRS10_drift = 0;
278 float old_gyrolPRS10 = 0;
279 float gyro9250_radians = 0;
280 float gyrolPRS10_radians = 0;
281
282 int16_t readdata[25];

```

```

283 int16_t IMU_data[9];
284
285 float accelx = 0;
286 float accely = 0;
287 float accelz = 0;
288 float gyrox = 0;
289 float gyroy = 0;
290 float gyroz = 0;
291
292 // Needed global Variables
293 float accelx_offset = 0;
294 float accely_offset = 0;
295 float accelz_offset = 0;
296 float gyrox_offset = 0;
297 float gyroy_offset = 0;
298 float gyroz_offset = 0;
299 int16_t calibration_state = 0;
300 int32_t calibration_count = 0;
301 int16_t doneCal = 0;
302
303 #define MPU9250 1
304 #define DAN28027 2
305 int16_t CurrentChip = MPU9250;
306 int16_t DAN28027Garbage = 0;
307 int16_t dan28027adc1 = 0;
308 int16_t dan28027adc2 = 0;
309 uint16_t MPU9250IgnoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
310 //Added variables to store camera-based ball distances - DS
311 float robotToBall2 = 0;
312 float robotToBall1 = 0;
313 float robotToBall3 = 0;
314
315 // angle of the hand encoder - ER
316 float RCangle = 0.0;
317 float colcentroid1 = 0.0;
318 float colcentroid2 = 0.0;
319 float colcentroid3 = 0.0;
320 float kpvision = -0.07; // vision proportional gain chosen so robot turns toward the ball centroid without overcorrecting - DS
321
322 int16_t count = 0;
323 int32_t statecount = 2000;
324 int16_t dwell=0;
325 float current= 0.0;
326 float target = 0.0;
327
328 void main(void)
329 {
330     // PLL, WatchDog, enable Peripheral Clocks
331     // This example function is found in the F2837xD_SysCtrl.c file.
332     InitSysCtrl();
333
334     InitGpio();
335
336     InitSE423DefaultGPIO();
337
338     //Scope for Timing
339     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
340     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSH_PULL);
341     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
342
343     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
344     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSH_PULL);
345     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
346
347     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
348     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSH_PULL);
349     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
350
351     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
352     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSH_PULL);
353     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
354
355     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
356     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSH_PULL);
357     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
358
359     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
360     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSH_PULL);
361     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
362
363     // ----- code for CAN start here -----
364     //GPIO17 - CANRXB
365     GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
366     GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
367
368     //GPIO12 - CANTXB
369     GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
370     GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSH_PULL);
371     // ----- code for CAN end here -----
372
373
374
375     // ----- code for CAN start here -----
376     // Initialize the CAN controller
377     InitCANB();
378
379     // Set up the CAN bus bit rate to 1000 kbps
380     setCANBitRate(200000000, 1000000);
381
382     // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
383     CanbRegs.CAN_CTL.bit.IE0= 1;
384     CanbRegs.CAN_CTL.bit.EIE= 1;
385     // ----- code for CAN end here -----
386
387     // Clear all interrupts and initialize PIE vector table:
388     // Disable CPU interrupts
389     DINT;
390
391     // Initialize the PIE control registers to their default state.
392     // The default state is all PIE interrupts disabled and flags
393     // are cleared.
394     // This function is found in the F2837xD_PieCtrl.c file.
395     InitPieCtrl();
396
397     // Disable CPU interrupts and clear all CPU interrupt flags:
398     IER = 0x0000;
399     IFR = 0x0000;
400
401     // Initialize the PIE vector table with pointers to the shell Interrupt
402     // Service Routines (ISR).
403     // This will populate the entire table, even if the interrupt
404     // is not used in this example. This is useful for debug purposes.
405     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
406     // This function is found in F2837xD_PieVect.c.
407     InitPieVectTable();
408
409     // Interrupts that are used in this example are re-mapped to
410     // ISR functions found within this project
411     EALLOW; // This is needed to write to EALLOW protected registers
412     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
413     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
414     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
415     PieVectTable.SCIA_RX_INT = &RXAINT_rcv_ready;
416     PieVectTable.SCIB_RX_INT = &RXBINT_rcv_ready;
417     PieVectTable.SCIC_RX_INT = &RXCINT_rcv_ready;
418     PieVectTable.SCID_RX_INT = &RXDINT_rcv_ready;
419     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
420     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
421     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
422     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
423     PieVectTable.ADCC1_INT = &ADCC_ISR;
424     PieVectTable.SPIB_RX_INT = &SPIB_isr;

```

```

425 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
426 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
427 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
428 // for ----- code for CAN start here -----
429 PieVectTable.CANB0_INT = &can_isr;
430 // ----- code for CAN end here -----
431 EDIS; // This is needed to disable write to EALLOW protected registers
432
433 // Initialize the CpuTimers Device Peripheral. This function can be
434 // found in F2837xD_CpuTimers.c
435 InitCpuTimers();
436
437 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
438 // 200MHz CPU Freq, Period (in uSeconds)
439 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
440 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
441 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
442
443 // Enable CpuTimer Interrupt bit TIE
444 CpuTimer0Regs.TCR.all = 0x4000;
445 CpuTimer1Regs.TCR.all = 0x4000;
446 CpuTimer2Regs.TCR.all = 0x4000;
447
448 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
449
450 init_serialSCIB(&SerialB,19200);
451 init_serialSCIC(&SerialC,19200);
452
453 for (LADARi = 0; LADARi < 228; LADARi++) {
454     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
455 }
456
457 init_eQEPs();
458 init_EPWM1and2();
459 init_RCServoPWM_3AB_5AB_6A();
460 init_ADCsAndDACs();
461 setupSpib();
462
463 EALLOW;
464 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
465 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
466 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
467 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
468 EPwm4Regs.TBCTR = 0x0; // Clear counter
469 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
470 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
471 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
472 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
473 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
474 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
475 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
476 EDIS;
477
478 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
479 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
480 // to CPU-Timer 2: int 12 is for the SWI.
481 IER |= M_INT1;
482 IER |= M_INT6;
483 IER |= M_INT8; // SCIC SCID
484 IER |= M_INT9; // SCIA
485 IER |= M_INT12;
486 IER |= M_INT13;
487 IER |= M_INT14;
488
489 // Enable TINT0 in the PIE: Group 1 interrupt 7
490 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
491 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
492 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
493 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
494 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
495 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
496 // ----- code for CAN start here -----
497 // Enable CANB in the PIE: Group 9 interrupt 7
498 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
499 // ----- code for CAN end here -----
500
501 // ----- code for CAN start here -----
502 // Enable the CAN interrupt signal
503 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
504 // ----- code for CAN end here -----
505
506 // 58.5 in * 7.5 tiles = 438.75 inches = 11.14425 meters
507
508
509 robotdest[0].x = 0; robotdest[0].y = 8*5;
510 robotdest[1].x = -8*5; robotdest[1].y = 8*5;
511 //middle of bottom
512 robotdest[2].x = 0; robotdest[2].y = 8*5;
513 // //outside the course
514 robotdest[3].x = 0; robotdest[3].y = 0;
515 // //back to middle
516 // robotdest[4].x = 0; robotdest[4].y = 2;
517 // robotdest[5].x = 4; robotdest[5].y = 2;
518 // robotdest[6].x = 4; robotdest[6].y = 10;
519 // robotdest[7].x = 0; robotdest[7].y = 9;
520 // // the two extra points that we added for the robot to go to after the initial course one in the middle of the field and one outside near the wall - ER
521 // robotdest[8].x = 2; robotdest[8].y = 5;
522 // robotdest[9].x = -2; robotdest[9].y = -1;
523
524 // ROBOTps will be updated by Optitrack during gyro calibration
525 // TODO: specify the starting position of the robot
526 ROBOTps.x = 0; //the estimate in array form (useful for matrix operations)
527 ROBOTps.y = 0;
528 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
529 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
530 x_pred[1][0] = ROBOTps.y;
531 x_pred[2][0] = ROBOTps.theta;
532
533 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
534 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
535 SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
536 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
537 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
538 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
539
540 init_serialSCIA(&SerialA,115200);
541 init_serialSCID(&SerialD,2083332);
542
543 // Enable global Interrupts and higher priority real-time debug events
544 EINT; // Enable Global interrupt INTM
545 ERTM; // Enable Global realtime interrupt DBGM
546 // ----- code for CAN start here -----
547 // Measured Distance from 1
548 // Initialize the receive message object 1 used for receiving CAN messages.
549 // Message Object Parameters:
550 // Message Object ID Number: 1
551 // Message Identifier: 0x060b0101
552 // Message Frame: Standard
553 // Message Type: Receive
554 // Message ID Mask: 0x0
555 // Message Object Flags: Receive Interrupt
556 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
557 // for a Receive mailbox)
558 //
559 CANsetuptMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
560 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
561 RX_MSG_DATA_LENGTH);
562
563 // Measured Distance from 2
564 // Initialize the receive message object 2 used for receiving CAN messages.
565 // Message Object Parameters:
566 // Message Object ID Number: 2

```

```

567 //      Message Identifier: 0x060b0102
568 //      Message Frame: Standard
569 //      Message Type: Receive
570 //      Message ID Mask: 0x0
571 //      Message Object Flags: Receive Interrupt
572 //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
573 //      for a Receive mailbox)
574 //
575
576 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
577                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
578                       RX_MSG_DATA_LENGTH);
579
580 // Measurement Quality from 1
581 // Initialize the receive message object 2 used for receiving CAN messages.
582 // Message Object Parameters:
583 //      Message Object ID Number: 3
584 //      Message Identifier: 0x060b0201
585 //      Message Frame: Standard
586 //      Message Type: Receive
587 //      Message ID Mask: 0x0
588 //      Message Object Flags: Receive Interrupt
589 //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
590 //      for a Receive mailbox)
591 //
592
593 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
594                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
595                       RX_MSG_DATA_LENGTH);
596
597 // Measurement Quality from 2
598 // Initialize the receive message object 2 used for receiving CAN messages.
599 // Message Object Parameters:
600 //      Message Object ID Number: 4
601 //      Message Identifier: 0x060b0202
602 //      Message Frame: Standard
603 //      Message Type: Receive
604 //      Message ID Mask: 0x0
605 //      Message Object Flags: Receive Interrupt
606 //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
607 //      for a Receive mailbox)
608 //
609
610 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
611                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
612                       RX_MSG_DATA_LENGTH);
613
614 //
615 // Start CAN module operations
616 //
617 CanbRegs.CAN_CTL.bit.Init = 0;
618 CanbRegs.CAN_CTL.bit.CCE = 0;
619
620
621 // ----- code for CAN end here -----
622 char S_command[19] = "S1152000124000\n";//this change the baud rate to 115200
623 uint16_t S_len = 19;
624 serial_sendSCIC(&SerialC, S_command, S_len);
625
626
627 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
628 init_serialSCIC(&SerialC, 115200);
629
630
631 // LED1 is GPIO22
632 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
633 // LED2 is GPIO94
634 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
635 // LED3 is GPIO95
636 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
637 // LED4 is GPIO97
638 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
639 // LED5 is GPIO111
640 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
641
642
643 // IDLE loop. Just sit and loop forever (optional):
644 while(1)
645 {
646     if (UARTPrint == 1 ) {
647         //UART_printfLine(1,"RCangle:%.2f",RCangle);
648         if (readbuttons() == 0) {
649             UART_printfLine(1,"RS: %d TC %ld", RobotState, checkfrontally);
650             //      UART_printfLine(1,"O1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold1,MaxColThreshold1,MaxRowThreshold1);
651             //      UART_printfLine(2,"PIA:%.0fC:%.0fR:%.0f",MaxAreaThreshold2,MaxColThreshold2,MaxRowThreshold2);
652             UART_printfLine(2, "LF %.2f SC %d", LADARfront, statecount);
653             //      UART_printfLine(1,"x:%.2f:y:%.2f:a%.2f",ROBOTps.x,ROBOTps.y,ROBOTps.theta);
654             //      UART_printfLine(2,"purple: %.2f", robotToBall12);
655         } else if (readbuttons() == 1) {
656             //      UART_printfLine(1,"O1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold1,MaxColThreshold1,MaxRowThreshold1);
657             //      UART_printfLine(2,"PIA:%.0fC:%.0fR:%.0f",MaxAreaThreshold2,MaxColThreshold2,MaxRowThreshold2);
658             //UART_printfLine(1,"LV1:%.3f LV2:%.3f",printLV1,printLV2);
659             //UART_printfLine(2,"Ln1:%.3f Ln2:%.3f",printLinux1,printLinux2);
660         } else if (readbuttons() == 2) {
661             UART_printfLine(1,"O2A:%.0fC:%.0fR:%.0f",NextLargestAreaThreshold1,NextLargestColThreshold1,NextLargestRowThreshold1);
662             UART_printfLine(2,"P2A:%.0fC:%.0fR:%.0f",NextLargestAreaThreshold2,NextLargestColThreshold2,NextLargestRowThreshold2);
663             // UART_printfLine(1,"%.2f %.2f",adcC2Volt,adcC3Volt);
664             // UART_printfLine(2,"%.2f %.2f",adcC4Volt,adcC5Volt);
665         } else if (readbuttons() == 4) {
666             UART_printfLine(1,"O3A:%.0fC:%.0fR:%.0f",NextNextLargestAreaThreshold1,NextNextLargestColThreshold1,NextNextLargestRowThreshold1);
667             UART_printfLine(2,"P3A:%.0fC:%.0fR:%.0f",NextNextLargestAreaThreshold2,NextNextLargestColThreshold2,NextNextLargestRowThreshold2);
668             // UART_printfLine(1,"L:%.3f R:%.3f",LeftVel,RightVel);
669             // UART_printfLine(2,"uL:%.2f uR:%.2f",uLeft,uRight);
670         } else if (readbuttons() == 8) {
671             UART_printfLine(1,"020x%.2f y%.2f",ladar_pts[20].x,ladar_pts[20].y);
672             UART_printfLine(2,"150x%.2f y%.2f",ladar_pts[150].x,ladar_pts[150].y);
673         } else if (readbuttons() == 3) {
674             UART_printfLine(1,"Vrf:%.2f trn:%.2f",vref,turn);
675             UART_printfLine(2,"MPU:%.2f LPR:%.2f",gyro9250_radians,gyroLPRS10_radians);
676         } else if (readbuttons() == 5) {
677             UART_printfLine(1,"0x:%.2f:0y:%.2f:0a%.2f",OPTITRACKps.x,OPTITRACKps.y,OPTITRACKps.theta);
678             UART_printfLine(2,"State:%d : %d",RobotState,statePos);
679         } else if (readbuttons() == 6) {
680             UART_printfLine(1,"D1 %ld D2 %ld",dis_1,dis_2);
681             UART_printfLine(2,"St1 %ld St2 %ld",measure_status_1,measure_status_2);
682         } else if (readbuttons() == 7) {
683             UART_printfLine(1,"%.0f,%.1f,%.1f,%.1f",tagid,tagx,tagy,tagz);
684             UART_printfLine(2,"%.1f,%.1f,%.1f",tagthetax,tagthetay,tagthetaz);
685         }
686     }
687     UARTPrint = 0;
688 }
689 }
690 }
691
692 __interrupt void SPIB_isr(void){
693
694     uint16_t i;
695     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
696
697     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
698     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
699
700     if (CurrentChip == MPU9250) {
701
702         for (i=0; i<8; i++) {
703             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
704         }
705
706         PostSWII(); // Manually cause the interrupt for the SWII
707
708     } else if (CurrentChip == DAN28027) {

```

```

709     DAN28027Garbage = SpibRegs.SPIRXBUF;
710     dan28027adc1 = SpibRegs.SPIRXBUF;
711     dan28027adc2 = SpibRegs.SPIRXBUF;
712     CurrentChip = MPU9250;
713     SpibRegs.SPIFFCT.bit.TXDLY = 0;
714 }
715
716
717 SpibRegs.SPIFFRX.bit.RXFFOVFLR=1; // Clear Overflow flag
718 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
719 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
720 GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
721 }
722
723
724 //adc1 pie interrupt
725 __interrupt void ADCC_ISR (void)
726 {
727     GpioDataRegs.GPASET.bit.GPIO11 = 1;
728     adcc2result = AdccResultRegs.ADCRESULT0;
729     adcc3result = AdccResultRegs.ADCRESULT1;
730     adcc4result = AdccResultRegs.ADCRESULT2;
731     adcc5result = AdccResultRegs.ADCRESULT3;
732
733     // Here covert ADCIND0 to volts
734     adc2Volt = adcc2result*3.0/4095.0;
735     adc3Volt = adcc3result*3.0/4095.0;
736     adc4Volt = adcc4result*3.0/4095.0;
737     adc5Volt = adcc5result*3.0/4095.0;
738
739     if (MPU9250ignoreCNT >= 1) {
740         CurrentChip = MPU9250;
741         SpibRegs.SPIFFCT.bit.TXDLY = 0;
742         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
743
744         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
745
746         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
747         SpibRegs.SPITXBUF = 0;
748         SpibRegs.SPITXBUF = 0;
749         SpibRegs.SPITXBUF = 0;
750         SpibRegs.SPITXBUF = 0;
751         SpibRegs.SPITXBUF = 0;
752         SpibRegs.SPITXBUF = 0;
753         SpibRegs.SPITXBUF = 0;
754     } else {
755         MPU9250ignoreCNT++;
756     }
757
758     numADCCcalls++;
759     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
760     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
761     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
762 }
763 }
764
765 // cpu_timer0_isr - CPU Timer0 ISR
766 __interrupt void cpu_timer0_isr(void)
767 {
768     CpuTimer0.InterruptCount++;
769
770     numTimer0calls++;
771
772     if ((numTimer0calls%5) == 0) {
773         // Blink LaunchPad Red LED
774         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
775     }
776
777     // Acknowledge this interrupt to receive more interrupts from group 1
778     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
779 }
780
781 // cpu_timer1_isr - CPU Timer1 ISR
782 __interrupt void cpu_timer1_isr(void)
783 {
784
785     serial_sendSCIC(&SerialC, G_command, G_len);
786
787     CpuTimer1.InterruptCount++;
788 }
789
790 // cpu_timer2_isr CPU Timer2 ISR
791 __interrupt void cpu_timer2_isr(void)
792 {
793     // Blink LaunchPad Blue LED
794     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
795     PostSWI3(); //Added PostSWI3 for Lab 7 Ex.1 Step 1 - DS
796     CpuTimer2.InterruptCount++;
797
798     // if ((CpuTimer2.InterruptCount % 10) == 0) {
799     //     UARTPrint = 1;
800     // }
801 }
802
803 void setF28027EPWM1A(float controleffort){
804     if (controleffort < -10) {
805         controleffort = -10;
806     }
807     if (controleffort > 10) {
808         controleffort = 10;
809     }
810     float value = (controleffort+10)*3000.0/20.0;
811     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
812 }
813 void setF28027EPWM2A(float controleffort){
814     if (controleffort < -10) {
815         controleffort = -10;
816     }
817     if (controleffort > 10) {
818         controleffort = 10;
819     }
820     float value = (controleffort+10)*3000.0/20.0;
821     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
822 }
823
824 //
825 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
826 //
827 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
828 {
829     GpioDataRegs.GPASET.bit.GPIO61 = 1;
830     // Set interrupt priority:
831     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
832     IER |= M_INT12;
833     IER &= MINT12; // Set "global" priority
834     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
835     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
836     asm(" NOP");
837     EINT;
838
839     //*****
840     IMU_data[0] = readdata[1];
841     IMU_data[1] = readdata[2];
842     IMU_data[2] = readdata[3];
843     IMU_data[3] = readdata[5];
844     IMU_data[4] = readdata[6];
845     IMU_data[5] = readdata[7];
846
847     accelx = (((float)(IMU_data[0]))*4.0/32767.0);
848     accely = (((float)(IMU_data[1]))*4.0/32767.0);
849     accelz = (((float)(IMU_data[2]))*4.0/32767.0);
850     gyrox = (((float)(IMU_data[3]))*250.0/32767.0);

```

```

851 gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
852 gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
853
854 if(calibration_state == 0){
855     calibration_count++;
856     if (calibration_count == 2000) {
857         calibration_state = 1;
858         calibration_count = 0;
859     }
860 } else if(calibration_state == 1){
861     accelx_offset+=accelx;
862     accely_offset+=accely;
863     accelz_offset+=accelz;
864     gyrox_offset+=gyrox;
865     gyroy_offset+=gyroy;
866     gyroz_offset+=gyroz;
867     gyroLPR510_offset+=adcC5Volt;
868     calibration_count++;
869     if (calibration_count == 2000) {
870         calibration_state = 2;
871         accelx_offset/=2000.0;
872         accely_offset/=2000.0;
873         accelz_offset/=2000.0;
874         gyrox_offset/=2000.0;
875         gyroy_offset/=2000.0;
876         gyroz_offset/=2000.0;
877         gyroLPR510_offset/=2000.0;
878         calibration_count = 0;
879     }
880 } else if(calibration_state == 2) {
881
882     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
883     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
884
885     gyro9250_drift += (((gyroz-gyro_offset) + old_gyro9250)*.0005)/(2000);
886     old_gyro9250 = gyroz-gyro_offset;
887     calibration_count++;
888
889     if (calibration_count == 2000) {
890         calibration_state = 3;
891         calibration_count = 0;
892         doneCal = 1;
893         newOPTITRACKpose = 0;
894     }
895 } else if(calibration_state == 3){
896     accelx -=(accelx_offset);
897     accely -=(accely_offset);
898     accelz -=(accelz_offset);
899     gyrox -= gyrox_offset;
900     gyroy -= gyroy_offset;
901     gyroz = gyro_offset;
902     LeftWheel = readEncLeft();
903     RightWheel = readEncRight();
904     HandValue = readEncWheel();
905
906     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
907     old_gyro9250 = gyroz;
908
909     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
910     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
911
912     gyro9250_radians = (gyro9250_angle * (PI/180.0));
913     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
914
915     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
916     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
917
918     if (newLinuxCommands == 1) {
919         newLinuxCommands = 0;
920         printLinux1 = LinuxCommands[0];
921         printLinux2 = LinuxCommands[1];
922         ///? = LinuxCommands[2];
923         ///? = LinuxCommands[3];
924         ///? = LinuxCommands[4];
925         ///? = LinuxCommands[5];
926         ///? = LinuxCommands[6];
927         ///? = LinuxCommands[7];
928         ///? = LinuxCommands[8];
929         ///? = LinuxCommands[9];
930         ///? = LinuxCommands[10];
931     }
932
933     if (NewCAMDataThreshold1 == 1) {
934         NewCAMDataThreshold1 = 0;
935         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
936         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
937         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
938
939         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
940         robotToBall1 = -0.00002154*MaxRowThreshold1*MaxRowThreshold1 + 0.006486*MaxRowThreshold1*MaxRowThreshold1 - 0.6728*MaxRowThreshold1 + 25.42;
941
942         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
943         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
944         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
945
946         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
947         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
948         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
949         numThres1++;
950         if ((numThres1 % 5) == 0) {
951             // LED4 is GPIO97
952             GpioDataRegs.GPDTGGLE.bit.GPIO97 = 1;
953         }
954     }
955
956     if (NewCAMDataThreshold2 == 1) {
957         NewCAMDataThreshold2 = 0;
958         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
959         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
960         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
961         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
962         if (ROBOTps.y < 35) { //first hallway
963             robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
964         }
965         else if (ROBOTps.y >= 35) { //second hallway
966             if (MaxRowThreshold2 > 40)
967                 robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
968             else if (MaxRowThreshold2 <= 40)
969                 robotToBall2 = 0;
970         }
971         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
972         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
973         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
974
975         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
976         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
977         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
978         numThres2++;
979         if ((numThres2 % 5) == 0) {
980             // LED5 is GPIO111
981             GpioDataRegs.GPDTGGLE.bit.GPIO111 = 1;
982         }
983     }
984
985     if (NewCAMDataThreshold3 == 1) {
986         NewCAMDataThreshold3 = 0;
987         MaxAreaThreshold3 = fromCAMvaluesThreshold3[0];
988         MaxColThreshold3 = fromCAMvaluesThreshold3[1];
989         MaxRowThreshold3 = fromCAMvaluesThreshold3[2];
990         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
991         robotToBall3 = -0.00002154*MaxRowThreshold3*MaxRowThreshold3*MaxRowThreshold3 + 0.006486*MaxRowThreshold3*MaxRowThreshold3 - 0.6728*MaxRowThreshold3 + 25.42;
992

```



```

993     NextLargestAreaThreshold3 = fromCAMvaluesThreshold3[3];
994     NextLargestColThreshold3 = fromCAMvaluesThreshold3[4];
995     NextLargestRowThreshold3 = fromCAMvaluesThreshold3[5];
996
997     NextNextLargestAreaThreshold3 = fromCAMvaluesThreshold3[6];
998     NextNextLargestColThreshold3 = fromCAMvaluesThreshold3[7];
999     NextNextLargestRowThreshold3 = fromCAMvaluesThreshold3[8];
1000     numThres3++;
1001     if ((numThres3 % 5) == 0) {
1002         // LED5 is GPIO111
1003         GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
1004     }
1005 }
1006
1007 if (NewCAMDataAprilTag1 == 1) {
1008     NewCAMDataAprilTag1 = 0;
1009     tagx = fromCAMvaluesAprilTag1[0];
1010     tagy = fromCAMvaluesAprilTag1[1];
1011     tagz = fromCAMvaluesAprilTag1[2];
1012
1013     tagthetax = fromCAMvaluesAprilTag1[3];
1014     tagthetay = fromCAMvaluesAprilTag1[4];
1015     tagthetaz = fromCAMvaluesAprilTag1[5];
1016
1017     tagid = fromCAMvaluesAprilTag1[6];
1018
1019     numtag++;
1020     if ((numtag % 5) == 0) {
1021         // LED2 is GPIO94
1022         GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
1023     }
1024 }
1025
1026 if (NewLVData == 1) {
1027     NewLVData = 0;
1028     printLV1 = fromLVvalues[0];
1029     printLV2 = fromLVvalues[1];
1030     //?? = fromLVvalues[2];
1031     //?? = fromLVvalues[3];
1032     //?? = fromLVvalues[4];
1033     //?? = fromLVvalues[5];
1034     //?? = fromLVvalues[6];
1035     //?? = fromLVvalues[7];
1036 }
1037
1038
1039 if (new_optitrack == 1) {
1040     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
1041     new_optitrack = 0;
1042 }
1043
1044 // Step 0: update B, u
1045 B[0][0] = cosf(ROBOTps.theta)*0.001;
1046 B[1][0] = sinf(ROBOTps.theta)*0.001;
1047 B[2][1] = 0.001;
1048 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
1049 u[1][0] = gyroZ*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
1050
1051 // Step 1: predict the state and estimate covariance
1052 Matrix3x2_Mult(B, u, Bu); // Bu = B*u
1053 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
1054 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
1055 // Step 2: if there is a new measurement, then update the state
1056 if (1 == newOPTITRACKpose) {
1057     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1058     if ((OptiNumUpdatesProcessed%10)==0) {
1059         // LED 3
1060         GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
1061     }
1062     newOPTITRACKpose = 0;
1063     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1064     z[1][0] = OPTITRACKps.y;
1065     // fix for OptiTrack problem at 180 degrees
1066     if (cosf(ROBOTps.theta) < -0.99) {
1067         z[2][0] = ROBOTps.theta;
1068     }
1069     else {
1070         z[2][0] = OPTITRACKps.theta;
1071     }
1072     // Step 2a: calculate the innovation/measurement residual, ytilde
1073     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1074     // Step 2b: calculate innovation covariance, S
1075     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
1076     // Step 2c: calculate the optimal Kalman gain, K
1077     Matrix3x3_Invert(S, S_inv);
1078     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1079     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1080     Matrix3x1_Mult(K, ytilde, temp_3x1);
1081     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1082     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1083     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1084     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1085 } // end of correction step
1086
1087 // set ROBOTps to the updated and corrected Kalman values.
1088 ROBOTps.x = x_pred[0][0];
1089 ROBOTps.y = x_pred[1][0];
1090 ROBOTps.theta = x_pred[2][0];
1091
1092 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1093 // uses xy code to step through an array of positions
1094 if (xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1095     statePos = (statePos+1)%NUMWAYPOINTS;
1096 }
1097
1098 dwell++;
1099 // state machine
1100 colcentroid1 = MaxColThreshold1 - 80;
1101 colcentroid2 = MaxColThreshold2 - 80; // 80 is used as the center column of the camera, so a centered green or orange ball centered gives centroid = 0 - DS
1102 colcentroid3 = MaxColThreshold3 - 80;
1103 switch (RobotState) {
1104 case 1: // command the robot to an X,Y point in the course - KL
1105     // vref and turn are the vref and turn returned from xy_control
1106     if (LADARfront < 1.2) {
1107         vref = 0.2;
1108         checkfronttally++;
1109         //DS: choose the side with more open space for obstacle avoidance
1110         if (checkfronttally > 300) {
1111             checkfronttally = 301; // no overflow error
1112             WallFollowtime = 0;
1113             if (LADARrightfront < LADARleftfront) {
1114                 RobotState = 10; //DS: use right-wall following
1115                 right_wall_follow_state = 1;
1116             } else {
1117                 RobotState = 12; //DS: use left-wall following
1118                 left_wall_follow_state = 1;
1119             }
1120         }
1121     } else {
1122         checkfronttally = 0;
1123     }
1124     // ensures that the robot does not move for 2 seconds after returning to state 1, so after is stops detecting a golf ball - ER
1125     statecount++;
1126     if (statecount<=2000){
1127         RobotState = 1;
1128     }else{
1129         statecount = 2001; // No overflow error
1130         count = 0;
1131         if (MaxAreaThreshold1 > 30) { // green ball detection threshold chosen so camera noise (image disturbance, not literal) does not trigger chasing rolling ball behavior - DS
1132             RobotState = 20;
1133             statecount = 0;
1134         }
1135     }

```



```

1135         if (MaxAreaThreshold2 > 30) { // same detection threshold but for orange ball - DS
1136             RobotState = 30;
1137             statecount = 0;
1138         }
1139         if (fabs(tagz) <= 350.468262) {
1140             if (tagid == 0.0){
1141                 RobotState = 99; //40;
1142             }
1143             if (tagid == 1.0){
1144                 RobotState = 50;
1145             }
1146         }
1147     }
1148 }
1149 break;
1150 case 10: // right wall following if an obstacle gets in the way of robot - KL
1151     if (right_wall_follow_state == 1) {
1152         //Left Turn
1153         turn = Kp_front_wall*(14.5 - LADARfront);
1154         vref = front_turn_velocity;
1155         if (LADARfront > left_turn_Stop_threshold) {
1156             right_wall_follow_state = 2;
1157         }
1158     } else if (right_wall_follow_state == 2) {
1159         //Right Wall Follow
1160         turn = Kp_right_wall*(ref_right_wall - LADARrightfront);
1161         vref = foward_velocity;
1162         if (LADARfront < left_turn_Start_threshold) {
1163             right_wall_follow_state = 1;
1164         }
1165     }
1166     if (turn > turn_saturation) {
1167         turn = turn_saturation;
1168     }
1169     if (turn < -turn_saturation) {
1170         turn = -turn_saturation;
1171     }
1172 }
1173 WallFollowtime++;
1174 // exit wall following if at least 5 seconds have passed and front path is clear - KL
1175 if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1176     RobotState = 1; //return to XY waypoint navigation - KL
1177     checkfronttally = 0;
1178 }
1179 break;
1180 case 12: //DS: left wall following if an obstacle gets in the way of robot
1181     if (left_wall_follow_state == 1) {
1182         //DS: Right Turn for left-wall following
1183         turn = -Kp_front_wall*(14.5 - LADARfront);
1184         vref = front_turn_velocity;
1185         if (LADARfront > right_turn_Stop_threshold) {
1186             left_wall_follow_state = 2;
1187         }
1188     } else if (left_wall_follow_state == 2) {
1189         //DS: Left Wall Follow
1190         turn = Kp_left_wall*(ref_left_wall - LADARleftfront);
1191         vref = foward_velocity;
1192         if (LADARfront < right_turn_Start_threshold) {
1193             left_wall_follow_state = 1;
1194         }
1195     }
1196     if (turn > turn_saturation) {
1197         turn = turn_saturation;
1198     }
1199     if (turn < -turn_saturation) {
1200         turn = -turn_saturation;
1201     }
1202 }
1203 }
1204 WallFollowtime++;
1205 //DS: exit left-wall following if at least 5 seconds have passed and front path is clear
1206 if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1207     RobotState = 1; //DS: return to XY waypoint navigation
1208     checkfronttally = 0;
1209 }
1210 break;
1211 // this state sets vref and turn so that the robot moves towards a detected green golf ball and after 1 second of waiting, sends the robot to state 22 which stops the robot - ER
1212 case 20:
1213     // put vision code here
1214     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3){
1215         vref = 0;
1216         turn = 0;
1217     } else {
1218         vref = 0.75;
1219         turn = kpvision * (0 - colcentroid1);
1220         if (MaxRowThreshold1 > 100){ //KLEC: row number for rb to know how to close it is to the ball, need to tune the number for enough clearance so that gate doesn't push ball away
1221             RobotState = 22;
1222             count = 0;
1223         }
1224     }
1225 }
1226 break;
1227 case 22: // after no longer detecting the green golf ball, the robot stops moving - ER, lift arm and move tongue
1228     vref = 0;
1229     turn = 0;
1230 }
1231 count++;
1232 if (count>=1000){//after 1 second, robot drives forward - KL
1233     RobotState = 24;
1234     count = 0;
1235 }
1236 break;
1237 case 24: // robot drives forward into the golf ball - KL
1238     vref = 0.5;
1239     turn = 0;
1240     setEPWM5B_RCServo(LTong); //KLEC: move the tongue to the left side, green balls go to the right compartment
1241     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1242     count++;
1243     if (count>=2000){
1244         RobotState = 26;
1245         count = 0;
1246     }
1247 }
1248 break;
1249 case 26: // robot pauses once ball is presumed to be captured - KL
1250     vref = 0;
1251     turn = 0;
1252     setEPWM6A_RCServo(Gate_C);
1253     setEPWM5B_RCServo(MTong);
1254     count++;
1255     if (count>=1000){ //after 1 second, robot returns to waypoint navigation - KL
1256         RobotState = 1;
1257         count = 0;
1258         statecount = 0;
1259     }
1260 }
1261 break;
1262 // states for orange ball
1263 case 30:
1264     // put vision code here
1265     //saying that if the orange ball is not detected or doesn't fit in the area threshold, then to stop the robot - KP
1266     //this helps it not detect other orange objects in the room as the ball - KP
1267     if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3){
1268         vref = 0;
1269         turn = 0;
1270     } else {
1271         //tells it what to do if it is detected, move to state 32 - KP
1272         vref = 0.75;
1273         turn = kpvision * (0 - colcentroid2);
1274         if (MaxRowThreshold2 > 100){
1275             RobotState = 32;
1276             count = 0;
1277         }
1278     }
1279 }
1280 }

```

```

1277     break;
1278
1279 case 32:
1280     //stopping and pausing with vref and turn at 0 - KP
1281     vref = 0;
1282     turn = 0;
1283     setEPWM5B_RCServo(RTong); //KLEC: move the tongue to the right side, orange balls go to the left compartment
1284     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1285     count++;
1286     if (count>=1000){
1287         RobotState = 34;
1288         count = 0;
1289     }
1290     break;
1291 case 34:
1292     //telling robot to move forward then move to state 36 - KP
1293     vref = 0.5;
1294     turn = 0;
1295     setEPWM5B_RCServo(RTong);
1296     setEPWM6A_RCServo(Gate_0);
1297     count++;
1298     if (count>=1000){
1299         RobotState = 36;
1300         count = 0;
1301     }
1302     break;
1303 case 36:
1304     //another stop before it begins the route again - KP
1305     vref = 0;
1306     turn = 0;
1307     setEPWM6A_RCServo(Gate_C);
1308     setEPWM5B_RCServo(MTong);
1309     count++;
1310     if (count>=1000){
1311         RobotState = 1;
1312         count = 0;
1313         statecount = 0;
1314     }
1315     break;
1316 // case 40://code telling the gate to open when it detects april tag 0
1317 // vref = 0;
1318 // turn = 0;
1319 //
1320 // setEPWM6A_RCServo(Gate_0);
1321 // setEPWM5B_RCServo(0);
1322 //
1323 // if (dwell >= 2000){
1324 //     setEPWM6A_RCServo(Gate_C);
1325 // }
1326 //
1327 // count++;
1328 // if (count>=1000){
1329 //     RobotState = 1;
1330 //     count = 0;
1331 //     statecount = 0;
1332 // }
1333 // break;
1334 case 50: //code telling the robot to turn
1335     vref = 0;
1336     count++;
1337     if (count>=2000){
1338         target = ROBOTps.theta + PI;
1339         RobotState = 52;
1340         count = 0;
1341     }
1342     break;
1343 case 52:
1344     turn = -(target - ROBOTps.theta);
1345     vref = 0;
1346     count++;
1347     if (fabs(target - ROBOTps.theta)<(PI/6)){
1348         statePos = 3;
1349         RobotState = 1;
1350         count = 0;
1351         statecount = 0;
1352         vref = 1.5;
1353     }
1354     break;
1355 case 99: //robot dispatching the ball // Entering condition: got back to the home April tag location
1356     vref = 0;
1357     turn = 0;
1358
1359     count++;
1360     //Drop off the color 1 green
1361     //EC: Tongue move to left location
1362     if (count >= 2000 && count < 4000){
1363         setEPWM5B_RCServo(LTong); //Lift arm
1364         setEPWM6A_RCServo(Gate_0);
1365         target = ROBOTps.theta + PI;
1366     } else if (count <= 7000){ //Back up fast for 3 seconds
1367         vref = -0.5;
1368     } else if (count <= 11000){
1369         vref = 0;
1370         setEPWM6A_RCServo(Gate_C);
1371         turn = -(target - ROBOTps.theta);
1372     } else if (count <= 13000){ //Go forward for 3 seconds
1373         vref = 0.5;
1374         turn = 0;
1375         setEPWM5B_RCServo(RTong);
1376     } else if (count <= 15000){ //Drop off the color 2 - red (move tongue to right)
1377         vref = 0;
1378         setEPWM6A_RCServo(Gate_0);
1379     } else if (count <= 19000){
1380         vref = -0.5;
1381     }
1382     else if (count <= 22000){
1383         setEPWM6A_RCServo(Gate_C); //Close the arm
1384         RobotState = 1;
1385         count = 0;
1386     }
1387     break;
1388
1389 default:
1390     break;
1391 }
1392
1393 //Must be called each time into this SWII function
1394 PControl(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1395 // These below lines also must be called each time into this SWII function
1396 setEPWM1A(uLeft);
1397 setEPWM2A(-uRight);
1398 setF20027EPWM1A(uLeft);
1399 setF20027EPWM2A(-uRight);
1400 LeftWheel_1 = LeftWheel;
1401 RightWheel_1 = RightWheel;
1402
1403 if((timecount%250) == 0) {
1404     DataToLabView.floatData[0] = ROBOTps.x;
1405     DataToLabView.floatData[1] = ROBOTps.y;
1406     DataToLabView.floatData[2] = ROBOTps.theta;
1407     DataToLabView.floatData[3] = (float)timecount;
1408     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1409     DataToLabView.floatData[5] = (float)RobotState;
1410     DataToLabView.floatData[6] = (float)statePos;
1411     DataToLabView.floatData[7] = LADARfront;
1412     LVsenddata[0] = '*'; // header for LVdata
1413     LVsenddata[1] = '$';
1414     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1415         if (i%2==0) {
1416             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1417         } else {
1418             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;

```

```

1419     }
1420     }
1421     serial_sendSCID(5SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1422 }
1423
1424 }
1425 timecount++;
1426 if((timecount%200) == 0)
1427 {
1428     if(doneCal == 0) {
1429         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1430     }
1431     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1432     UARTPrint = 1; // Tell While loop to print
1433 }
1434
1435 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1436 CurrentChip = DAN28027;
1437 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1438 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1439 SpibRegs.SPITXBUF = 0x00DA;
1440 SpibRegs.SPITXBUF = EPwm1A_F28027;
1441 SpibRegs.SPITXBUF = EPwm2A_F28027;
1442
1443 //#####
1444 //
1445 // Restore registers saved:
1446 //
1447 DINT;
1448 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1449
1450 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1451 }
1452
1453 //
1454 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1455 //
1456 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1457 {
1458     // Set interrupt priority:
1459     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1460     IER |= M_INT12;
1461     IER &= MINT12; // Set "global" priority
1462     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1463     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1464     __asm(" NOP");
1465     EINT;
1466
1467 //#####
1468 // Insert SWI ISR Code here.....
1469 // LED1
1470 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1471 if (LADARpingpong == 1) {
1472     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1473     LADARrightfront = 19; // 19 is greater than max feet
1474     for (LADARi = 52; LADARi <= 56; LADARi++) {
1475         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1476             LADARrightfront = ladar_data[LADARi].distance_ping;
1477         }
1478     }
1479     //DS: LADARleftfront is the min of dist 170, 171, 172, 173, 174
1480     LADARleftfront = 19;
1481     for (LADARi = 170; LADARi <= 174; LADARi++) {
1482         if (ladar_data[LADARi].distance_ping < LADARleftfront) {
1483             LADARleftfront = ladar_data[LADARi].distance_ping;
1484         }
1485     }
1486     // LADARfront is the min of dist 111, 112, 113, 114, 115
1487     LADARfront = 19;
1488     for (LADARi = 111; LADARi <= 115; LADARi++) {
1489         if (ladar_data[LADARi].distance_ping < LADARfront) {
1490             LADARfront = ladar_data[LADARi].distance_ping;
1491         }
1492     }
1493     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1494     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1495     for (LADARi = 0; LADARi < 228; LADARi++) {
1496
1497         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1498         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1499     }
1500 } else if (LADARpingpong == 0) {
1501     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1502     LADARrightfront = 19; // 19 is greater than max feet
1503     for (LADARi = 52; LADARi <= 56; LADARi++) {
1504         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1505             LADARrightfront = ladar_data[LADARi].distance_pong;
1506         }
1507     }
1508     //DS: LADARleftfront is the min of dist 170, 171, 172, 173, 174
1509     LADARleftfront = 19;
1510     for (LADARi = 170; LADARi <= 174; LADARi++) {
1511         if (ladar_data[LADARi].distance_pong < LADARleftfront) {
1512             LADARleftfront = ladar_data[LADARi].distance_pong;
1513         }
1514     }
1515     // LADARfront is the min of dist 111, 112, 113, 114, 115
1516     LADARfront = 19;
1517     for (LADARi = 111; LADARi <= 115; LADARi++) {
1518         if (ladar_data[LADARi].distance_pong < LADARfront) {
1519             LADARfront = ladar_data[LADARi].distance_pong;
1520         }
1521     }
1522     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1523     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1524     for (LADARi = 0; LADARi < 228; LADARi++) {
1525
1526         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1527         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1528     }
1529 }
1530 }
1531 }
1532
1533 //#####
1534 //
1535 // Restore registers saved:
1536 //
1537 DINT;
1538 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1539
1540 }
1541
1542 //
1543 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1544 //
1545 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1546 {
1547     // Set interrupt priority:
1548     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1549     IER |= M_INT12;
1550     IER &= MINT12; // Set "global" priority
1551     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1552     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1553     __asm(" NOP");
1554     EINT;
1555     RCangle = readEncWheel(); //Lab RC servo exercise uses encoder angle as the command input for servo motion - DS
1556     // setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1557     // setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1558     // setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use
1559     // setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Tongue

```

```

1561 //      setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Gate
1562 //#####
1563 // Insert SWI ISR Code here.....
1564
1565 //#####
1566 //
1567 // Restore registers saved:
1568 //
1569 DINT;
1570 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1571
1572 }
1573
1574 // ----- code for CAN start here -----
1575 __interrupt void can_isr(void)
1576 {
1577     int i = 0;
1578
1579     uint32_t status;
1580
1581     //
1582     // Read the CAN interrupt status to find the cause of the interrupt
1583     //
1584     status = CANGetInterruptCause(CANB_BASE);
1585
1586     //
1587     // If the cause is a controller status interrupt, then get the status
1588     //
1589     if(status == CAN_INT_INT0ID_STATUS)
1590     {
1591         //
1592         // Read the controller status. This will return a field of status
1593         // error bits that can indicate various errors. Error processing
1594         // is not done in this example for simplicity. Refer to the
1595         // API documentation for details about the error status bits.
1596         // The act of reading this status will clear the interrupt.
1597         //
1598         status = CANGetStatus(CANB_BASE);
1599     }
1600
1601
1602     //
1603     // Check if the cause is the receive message object 2
1604     //
1605     else if(status == RX_MSG_OBJ_ID_1)
1606     {
1607         //
1608         // Get the received message
1609         //
1610         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1611
1612         for(i = 0; i<2; i++)
1613         {
1614             dis_raw_1[i] = rxMsgData[i];
1615         }
1616
1617         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1618
1619         measure_status_1 = rxMsgData[2];
1620
1621         //
1622         // Getting to this point means that the RX interrupt occurred on
1623         // message object 2, and the message RX is complete. Clear the
1624         // message object interrupt.
1625         //
1626         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1627
1628         //
1629         // Increment a counter to keep track of how many messages have been
1630         // received. In a real application this could be used to set flags to
1631         // indicate when a message is received.
1632         //
1633         rxMsgCount_1++;
1634
1635         //
1636         // Since the message was received, clear any error flags.
1637         //
1638         errorFlag = 0;
1639     }
1640
1641     else if(status == RX_MSG_OBJ_ID_2)
1642     {
1643         //
1644         // Get the received message
1645         //
1646         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1647
1648         for(i = 0; i<2; i++)
1649         {
1650             dis_raw_2[i] = rxMsgData[i];
1651         }
1652
1653         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1654
1655         measure_status_2 = rxMsgData[2];
1656
1657         //
1658         // Getting to this point means that the RX interrupt occurred on
1659         // message object 2, and the message RX is complete. Clear the
1660         // message object interrupt.
1661         //
1662         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1663
1664         //
1665         // Increment a counter to keep track of how many messages have been
1666         // received. In a real application this could be used to set flags to
1667         // indicate when a message is received.
1668         //
1669         rxMsgCount_2++;
1670
1671         //
1672         // Since the message was received, clear any error flags.
1673         //
1674         errorFlag = 0;
1675     }
1676
1677     else if(status == RX_MSG_OBJ_ID_3)
1678     {
1679         //
1680         // Get the received message
1681         //
1682         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1683
1684         for(i = 0; i<4; i++)
1685         {
1686             lightlevel_raw_1[i] = rxMsgData[i];
1687             quality_raw_1[i] = rxMsgData[i+4];
1688         }
1689
1690         lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1691         quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1692
1693
1694         //
1695         // Getting to this point means that the RX interrupt occurred on
1696         // message object 2, and the message RX is complete. Clear the
1697         // message object interrupt.
1698         //
1699         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1700
1701         //
1702

```

```

1703         // Since the message was received, clear any error flags.
1704         //
1705         errorFlag = 0;
1706     }
1707
1708
1709     else if(status == RX_MSG_OBJ_ID_4)
1710     {
1711         //
1712         // Get the received message
1713         //
1714         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1715
1716         for(i = 0; i<4; i++)
1717         {
1718             lightlevel_raw_2[i] = rxMsgData[i];
1719             quality_raw_2[i] = rxMsgData[i+4];
1720         }
1721
1722
1723         lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1724         quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1725
1726         //
1727         // Getting to this point means that the RX interrupt occurred on
1728         // message object 2, and the message RX is complete. Clear the
1729         // message object interrupt.
1730         //
1731         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1732
1733         //
1734         // Since the message was received, clear any error flags.
1735         //
1736         errorFlag = 0;
1737     }
1738
1739
1740
1741     //
1742     // If something unexpected caused the interrupt, this would handle it.
1743     //
1744     else
1745     {
1746         //
1747         // Spurious interrupt handling can go here.
1748         //
1749     }
1750
1751     //
1752     // Clear the global interrupt flag for the CAN interrupt line
1753     //
1754     CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1755
1756     //
1757     // Acknowledge this interrupt located in group 9
1758     //
1759     InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1760 }
1761 // ----- code for CAN end here -----

```

```

1  /* SERIAL.C: This code is designed to act as a low-level serial driver for
2     higher-level programming. Ideally, one could simply call init_serial()
3     to initialize the serial port, then use serial_send("data", 4) to send
4     an array of data (8-bit unsigned character strings).
5
6     WRITTEN BY : Paul Miller <pamiller@uiuc.edu>
7     $Id: serial.c,v 1.4 2003/08/08 16:08:56 paul Exp $
8  */
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15
16 #include "F28x_Project.h"      // Device Headerfile and Examples Include File
17 #include <buffer.h>
18 #include <F28379dSerial.h>
19 #include <F2837xD_sci.h>
20 #include "SE423Lib.h"
21
22 serialSCIA_t SerialA;
23 serialSCIB_t SerialB;
24 serialSCIC_t SerialC;
25 serialSCID_t SerialD;
26
27 char RXAdata = 0;
28 char RXBdata = 0;
29 char RxCdata = 0;
30 char RxDdata = 0;
31 uint32_t numRXA = 0;
32 uint32_t numRXB = 0;
33 uint32_t numRXC = 0;
34 uint32_t numRXD = 0;
35
36 char buf[477]; //store all the data
37 uint16_t dis_index = 0;
38 uint16_t G_count = 0; //number of thing received after 'G'
39 uint16_t LF_count = 0; //number of consecutive line feed
40 uint16_t buf_index = 0; //for debug
41 uint16_t get_dis_first; //if it is the first part of the two parts data
42 uint16_t print_dis_index=0;
43 uint32_t numRCXerrors = 0;
44 int16_t tempscid = 0;
45 int16_t distance = 0;
46 float checkdist = 0;
47
48 uint16_t COMA_state = 0;
49 uint16_t COMALSB = 0;
50
51 uint16_t received_CAM_countThreshold1 = 0;
52 uint16_t NewCAMDataThreshold1 = 0; // Flag new data
53 CAMRecFloats_t DataFromCameraThreshold1;
54 float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
55
56 uint16_t received_CAM_countThreshold2 = 0;
57 uint16_t NewCAMDataThreshold2 = 0; // Flag new data
58 CAMRecFloats_t DataFromCameraThreshold2;
59 float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
60
61 //EC: Declaration added for the third color
62 uint16_t received_CAM_countThreshold3 = 0;
63 uint16_t NewCAMDataThreshold3 = 0; // Flag new data
64 CAMRecFloats_t DataFromCameraThreshold3;
65 float fromCAMvaluesThreshold3[CAMNUM_FROM_FLOATS];
66
67 uint16_t received_CAM_countAprilTag1 = 0;
68 uint16_t NewCAMDataAprilTag1 = 0; // Flag new data
69 CAMRecFloats_t DataFromCameraAprilTag1;
70 float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
71
72 uint16_t tempLSB = 0;
73 LVRecFloats_t DataFromLabView;
74 LVSendFloats_t DataToLabView;
75 uint16_t received_LV_count = 0;
76 uint16_t NewLVData = 0;
77 char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
78 float fromLVvalues[LVNUM_TOFROM_FLOATS];
79
80 CMDRecFloats_t DataFromLinuxCMD;
81 uint16_t received_CMD_count = 0;
82 uint16_t newLinuxCommands = 0;
83 float LinuxCommands[CMDNUM_FROM_FLOATS];
84 uint16_t com_state = 0; //the state of receive from pi (SCID)
85 uint16_t LADARpingpong = 0;
86 uint16_t read_ladar_state = 0; //states for ladar reading, initialized as wait for S command
87 datapts_ladar_data[228]; //distance data from ladar
88

```

```

89 uint16_t new_optitrack = 0;
90 float Optitrackdata[OPTITRACKDATASIZE]; // x, y, heading, trackableID, framecount
91 uint16_t received_opti_count = 0; // the count for receiving optitrack data
92 optiData_t optirx;
93
94 char path_received[81];
95 int16_t received_path_count = 0;
96 int16_t newAstarPath = 0;
97 float ladar161distance = 0;
98 float ladar199distance = 0;
99
100
101 // for SerialA
102 uint16_t init_serialSCIA(serialSCIA_t *s, uint32_t baud)
103 {
104     volatile struct SCI_REGS *sci;
105     uint32_t clk;
106
107     if (s == &SerialA) {
108         sci = &SciaRegs;
109         s->sci = sci;
110
111         init_bufferSCIA(&s->TX);
112
113         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 0);
114         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
115         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 0);
116         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
117
118     } else {
119         return 1;
120     }
121
122     /* init for standard baud, 8N1 comm */
123     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
124     sci->SCICCR.all = 0x0;
125     sci->SCICTL1.all = 0x0;
126     sci->SCICTL2.all = 0x0;
127     sci->SCIPRI.all = 0x0;
128     clk = LSPCLK_HZ; // set baud rate
129     clk /= baud*8;
130     clk--;
131     sci->SCILBAUD.all = clk & 0xFF;
132     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
133
134     sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
135     sci->SCICCR.bit.PARIYENA = 0; // (N) disable parity calculation
136     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
137     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
138     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
139
140     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
141
142     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
143     sci->SCIFFTX.bit.TXFIFORESET = 0;
144     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
145     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
146     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
147     sci->SCIFFTX.bit.TXFIFORESET = 1;
148
149     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
150     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
151     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
152     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
153     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
154
155     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
156     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
157
158     sci->SCICTL1.bit.TXWAKE = 0;
159     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
160     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
161     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
162     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
163     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
164
165     /* enable PIE interrupts */
166     if (s == &SerialA) {
167         PieCtrlRegs.PIEIER9.bit.INTx1 = 1;
168         PieCtrlRegs.PIEIER9.bit.INTx2 = 1;
169         IER |= (M_INT9);
170         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
171         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 15);
172         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
173         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 15);
174         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
175
176     }
177
178     return 0;
179 }

```



```

180
181 void uninit_serialSCIA(serialSCIA_t *s)
182 {
183     volatile struct SCI_REGS *sci = s->sci;
184
185     /* disable PIE interrupts */
186     if (s == &SerialA) {
187         PieCtrlRegs.PIEIER9.bit.INTx1 = 0;
188         PieCtrlRegs.PIEIER9.bit.INTx2 = 0;
189         IER &= ~M_INT9;
190     }
191     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
192     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
193     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
194
195     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
196     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
197 }
198
199
200
201 /*****
202  * SERIAL_SEND()
203  *
204  * "User level" function to send data via serial. Return value is the
205  * length of data successfully copied to the TX buffer.
206  *****/
207 uint16_t serial_sendSCIA(serialSCIA_t *s, char *data, uint16_t len)
208 {
209     uint16_t i = 0;
210     if (len && s->TX.size < BUF_SIZESCIA) {
211         for (i = 0; i < len; i++) {
212             if (buf_writeSCIA_1(&s->TX, data[i] & 0x00FF) != 0) break;
213         }
214         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
215         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
216     }
217     return i;
218 }
219
220 // For SerialB
221 uint16_t init_serialSCIB(serialSCIB_t *s, uint32_t baud)
222 {
223     volatile struct SCI_REGS *sci;
224     uint32_t clk;
225
226     if (s == &SerialB) {
227         sci = &ScibRegs;
228         s->sci = sci;
229
230         init_bufferSCIB(&s->TX);
231
232         GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 0);
233         GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
234         GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 0);
235         GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
236
237     } else {
238         return 1;
239     }
240 }
241
242 /* init for standard baud, 8N1 comm */
243 sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
244 sci->SCICCR.all = 0x0;
245 sci->SCICTL1.all = 0x0;
246 sci->SCICTL2.all = 0x0;
247 sci->SCIPRI.all = 0x0;
248 clk = LSPCLK_HZ; // set baud rate
249 clk /= baud*8;
250 clk--;
251 sci->SCILBAUD.all = clk & 0xFF;
252 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
253
254 sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
255 sci->SCICCR.bit.PARIYENA = 0; // (N) disable parity calculation
256 sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
257 sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
258 sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
259
260 sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
261
262 sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
263 sci->SCIFFTX.bit.TXFIFORESET = 0;
264 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
265 sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
266 sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
267 sci->SCIFFTX.bit.TXFIFORESET = 1;
268
269 sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
270 sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag

```

```

271 sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
272 sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
273 sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
274
275 sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
276 sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
277
278 sci->SCICTL1.bit.TXWAKE = 0;
279 sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
280 sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
281 sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
282 sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
283 sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
284
285 /* enable PIE interrupts */
286 if (s == &SerialB) {
287     PieCtrlRegs.PIEIER9.bit.INTx3 = 1;
288     PieCtrlRegs.PIEIER9.bit.INTx4 = 1;
289     IER |= (M_INT9);
290     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
291     GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 2);
292     GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
293     GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 2);
294     GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
295 }
296
297
298 return 0;
299 }
300
301 void uninit_serialSCIB(serialSCIB_t *s)
302 {
303     volatile struct SCI_REGS *sci = s->sci;
304
305     /* disable PIE interrupts */
306     if (s == &SerialB) {
307         PieCtrlRegs.PIEIER9.bit.INTx3 = 0;
308         PieCtrlRegs.PIEIER9.bit.INTx4 = 0;
309         IER &= ~M_INT9;
310     }
311
312     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
313     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
314     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
315
316     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
317     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
318 }
319
320
321
322 /*****
323  * SERIAL_SEND()
324  *
325  * "User level" function to send data via serial. Return value is the
326  * length of data successfully copied to the TX buffer.
327  *****/
328 uint16_t serial_sendSCIB(serialSCIB_t *s, char *data, uint16_t len)
329 {
330     uint16_t i = 0;
331     if (len && s->TX.size < BUF_SIZE_SCIB) {
332         for (i = 0; i < len; i++) {
333             if (buf_writeSCIB_1(&s->TX, data[i] & 0x00FF) != 0) break;
334         }
335         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
336         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
337     }
338     return i;
339 }
340
341 // for SerialC
342 uint16_t init_serialSCIC(serialSCIC_t *s, uint32_t baud)
343 {
344     volatile struct SCI_REGS *sci;
345     uint32_t clk;
346
347     if (s == &SerialC) {
348         sci = &ScicRegs;
349         s->sci = sci;
350
351         init_bufferSCIC(&s->TX);
352         GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 0);
353         GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
354         GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 0);
355         GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
356
357     } else {
358         return 1;
359     }
360
361     /* init for standard baud, 8N1 comm */

```

```

362 sci->SCICTL1.bit.SWRESET = 0;          // init SCI state machines and opt flags
363 sci->SCICCR.all = 0x0;
364 sci->SCICTL1.all = 0x0;
365 sci->SCICTL2.all = 0x0;
366 sci->SCIPRI.all = 0x0;
367 clk = LSPCLK_HZ;                      // set baud rate
368 clk /= baud*8;
369 clk--;
370 sci->SCILBAUD.all = clk & 0xFF;
371 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
372
373 sci->SCICCR.bit.SCICHAR = 0x7;          // (8) 8 bits per character
374 sci->SCICCR.bit.PARITYENA = 0;          // (N) disable parity calculation
375 sci->SCICCR.bit.STOPBITS = 0;           // (1) transmit 1 stop bit
376 sci->SCICCR.bit.LOOPBKENA = 0;          // disable loopback test
377 sci->SCICCR.bit.ADDRIDLE_MODE = 0;      // idle-line mode (non-multiprocessor SCI comm)
378
379 sci->SCIFFCT.bit.FFTXDLY = 0;           // TX: zero-delay
380
381 sci->SCIFFTX.bit.SCIFFENA = 1;           // enable SCI fifo enhancements
382 sci->SCIFFTX.bit.TXFIFORESET = 0;
383 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels  ??? is this correct
384 sci->SCIFFTX.bit.TXFFINTCLR = 1;        // TX: clear interrupt flag
385 sci->SCIFFTX.bit.TXFFIENA = 0;          // TX: disable fifo interrupt
386 sci->SCIFFTX.bit.TXFIFORESET = 1;
387
388 sci->SCIFFRX.bit.RXFIFORESET = 0;        // RX: fifo reset
389 sci->SCIFFRX.bit.RXFFINTCLR = 1;        // RX: clear interrupt flag
390 sci->SCIFFRX.bit.RXFFIENA = 1;          // RX: enable fifo interrupt
391 sci->SCIFFRX.bit.RXFFIL = 0x1;          // RX: fifo interrupt
392 sci->SCIFFRX.bit.RXFIFORESET = 1;        // RX: re-enable fifo
393
394 sci->SCICTL2.bit.RXBKINTENA = 0;         // disable receiver/error interrupt
395 sci->SCICTL2.bit.TXINTENA = 0;          // disable transmitter interrupt
396
397 sci->SCICTL1.bit.TXWAKE = 0;
398 sci->SCICTL1.bit.SLEEP = 0;             // disable sleep mode
399 sci->SCICTL1.bit.RXENA = 1;             // enable SCI receiver
400 sci->SCICTL1.bit.RXERRINTENA = 0;       // disable receive error interrupt
401 sci->SCICTL1.bit.TXENA = 1;             // enable SCI transmitter
402 sci->SCICTL1.bit.SWRESET = 1;           // re-enable SCI
403
404 /* enable PIE interrupts */
405 if (s == &SerialC) {
406     PieCtrlRegs.PIEIER8.bit.INTx5 = 1;
407     PieCtrlRegs.PIEIER8.bit.INTx6 = 1;
408     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
409     IER |= (M_INT8);
410     GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 6);
411     GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
412     GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 6);
413     GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
414 }
415
416 return 0;
417 }
418
419 void uninit_serialSCIC(serialSCIC_t *s)
420 {
421     volatile struct SCI_REGS *sci = s->sci;
422
423     /* disable PIE interrupts */
424     if (s == &SerialC) {
425         PieCtrlRegs.PIEIER8.bit.INTx5 = 0;
426         PieCtrlRegs.PIEIER8.bit.INTx6 = 0;
427         IER &= ~M_INT8;
428     }
429
430     sci->SCICTL1.bit.RXERRINTENA = 0;     // disable receive error interrupt
431     sci->SCICTL2.bit.RXBKINTENA = 0;     // disable receiver/error interrupt
432     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
433
434     sci->SCICTL1.bit.RXENA = 0;           // disable SCI receiver
435     sci->SCICTL1.bit.TXENA = 0;           // disable SCI transmitter
436 }
437
438
439
440 /*****
441  * SERIAL_SEND()
442  *
443  * "User level" function to send data via serial. Return value is the
444  * length of data successfully copied to the TX buffer.
445  *****/
446 uint16_t serial_sendSCIC(serialSCIC_t *s, char *data, uint16_t len)
447 {
448     uint16_t i = 0;
449     if (len && s->TX.size < BUF_SIZE_SCIC) {
450         for (i = 0; i < len; i++) {
451             if (buf_writeSCIC_1(&s->TX, data[i] & 0x00FF) != 0) break;
452         }
453     }

```

```

453     s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
454     s->sci->SCIFFTX.bit.TXFFIENA = 1;    // TX: enable fifo interrupt
455 }
456 return i;
457 }
458
459 // For Serial D
460 uint16_t init_serialSCID(serialSCID_t *s, uint32_t baud)
461 {
462     volatile struct SCI_REGS *sci;
463     uint32_t clk;
464
465     if (s == &SerialD) {
466         sci = &ScidRegs;
467         s->sci = sci;
468
469         init_bufferSCID(&s->TX);
470         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 0);
471         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
472         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 0);
473         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
474     }
475     else {
476         return 1;
477     }
478
479     /* init for standard baud,8N1 comm */
480     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
481     sci->SCICCR.all = 0x0;
482     sci->SCICTL1.all = 0x0;
483     sci->SCICTL2.all = 0x0;
484     sci->SCIPRI.all = 0x0;
485     clk = LSPCLK_HZ; // set baud rate
486     clk /= baud*8;
487     clk--;
488     sci->SCILBAUD.all = clk & 0xFF;
489     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
490
491     sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
492     sci->SCICCR.bit.PARITYENA = 0; // (N) disable parity calculation
493     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
494     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
495     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
496
497     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
498
499     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
500     sci->SCIFFTX.bit.TXFIFORESET = 0;
501     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
502     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
503     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
504     sci->SCIFFTX.bit.TXFIFORESET = 1;
505
506     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
507     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
508     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
509     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
510     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
511
512     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
513     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
514
515     sci->SCICTL1.bit.TXWAKE = 0;
516     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
517     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
518     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
519     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
520     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
521
522     /* enable PIE interrupts */
523     if (s == &SerialD) {
524         PieCtrlRegs.PIEIER8.bit.INTx7 = 1;
525         PieCtrlRegs.PIEIER8.bit.INTx8 = 1;
526         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
527         IER |= (M_INT8);
528         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 6);
529         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
530         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 6);
531         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
532     }
533 }
534
535 return 0;
536 }
537
538 void uninit_serialSCID(serialSCID_t *s)
539 {
540     volatile struct SCI_REGS *sci = s->sci;
541
542     /* disable PIE interrupts */
543     if (s == &SerialD) {

```

```

544     PieCtrlRegs.PIEIER8.bit.INTx7 = 0;
545     PieCtrlRegs.PIEIER8.bit.INTx8 = 0;
546     IER &= ~M_INT8;
547 }
548
549 sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
550 sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
551 sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
552
553 sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
554 sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
555 }
556
557
558
559 /*****
560  * SERIAL_SEND()
561  *
562  * "User level" function to send data via serial. Return value is the
563  * length of data successfully copied to the TX buffer.
564  *****/
565 uint16_t serial_sendSCID(serialSCID_t *s, char *data, uint16_t len)
566 {
567     uint16_t i = 0;
568     if (len && s->TX.size < BUF_SIZE_SCID) {
569         for (i = 0; i < len; i++) {
570             if (buf_writeSCID_1(&s->TX, data[i] & 0x00FF) != 0) break;
571         }
572         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
573         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
574     }
575     return i;
576 }
577
578
579
580 /*****
581  * TXxINT_DATA_SENT()
582  *
583  * Executed when transmission is ready for additional data. These functions
584  * read the next char of data and put it in the TXBUF register for transfer.
585  *****/
586 #ifdef _FLASH
587 #pragma CODE_SECTION(TXAINT_data_sent, ".TI.ramfunc");
588 #endif
589 __interrupt void TXAINT_data_sent(void)
590 {
591     char data;
592     if (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0) {
593         while ( (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0)
594             && (SerialA.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
595             buf_removeSCIA(&SerialA.TX, 1);
596             SerialA.sci->SCITXBUF.all = data;
597         }
598     } else {
599         SerialA.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
600     }
601     SerialA.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
602     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
603 }
604
605
606 //for serialB
607 #ifdef _FLASH
608 #pragma CODE_SECTION(TXBINT_data_sent, ".TI.ramfunc");
609 #endif
610 __interrupt void TXBINT_data_sent(void)
611 {
612     char data;
613     if (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0) {
614         while ( (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0)
615             && (SerialB.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
616             buf_removeSCIB(&SerialB.TX, 1);
617             SerialB.sci->SCITXBUF.all = data;
618         }
619     } else {
620         SerialB.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
621     }
622     SerialB.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
623     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
624 }
625
626
627 //for serialC
628 #ifdef _FLASH
629 #pragma CODE_SECTION(TXCINT_data_sent, ".TI.ramfunc");
630 #endif
631 __interrupt void TXCINT_data_sent(void)
632 {
633     char data;
634     if (buf_readSCIC_1(&SerialC.TX, 0, &data) == 0) {

```

```

635     while ( (buf_readSCIC_1(&SerialC.TX,0,&data) == 0)
636             && (SerialC.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
637         buf_removeSCIC(&SerialC.TX, 1);
638         SerialC.sci->SCITXBUF.all = data;
639     }
640 } else {
641     SerialC.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
642 }
643 SerialC.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
644 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
645 }
646
647 //for serialD
648 #ifdef _FLASH
649 #pragma CODE_SECTION(TXDINT_data_sent, ".TI.ramfunc");
650 #endif
651 __interrupt void TXDINT_data_sent(void)
652 {
653     char data;
654     if (buf_readSCID_1(&SerialD.TX,0,&data) == 0) {
655         while ( (buf_readSCID_1(&SerialD.TX,0,&data) == 0)
656                 && (SerialD.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
657             buf_removeSCID(&SerialD.TX, 1);
658             SerialD.sci->SCITXBUF.all = data;
659         }
660     } else {
661         SerialD.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
662     }
663     SerialD.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
664     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
665 }
666
667 //for SerialA
668 #ifdef _FLASH
669 #pragma CODE_SECTION(RXAINT_recv_ready, ".TI.ramfunc");
670 #endif
671 __interrupt void RXAINT_recv_ready(void)
672 {
673     RXAdata = SciaRegs.SCIRXBUF.all;
674
675     /* SCI PE or FE error */
676     if (RXAdata & 0xC000) {
677         SciaRegs.SCICTL1.bit.SWRESET = 0;
678         SciaRegs.SCICTL1.bit.SWRESET = 1;
679         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
680         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;
681     } else {
682         RXAdata = RXAdata & 0x00FF;
683         if (COMA_state == 0) {
684
685             if (RXAdata == '*') {
686                 COMA_state = 1; //for receiving path
687             } else {
688                 COMA_state = 0;
689             }
690         } else if (COMA_state == 1) {
691
692             if (RXAdata == '*') {
693                 COMA_state = 10; // into the read mode
694                 received_CAM_countThreshold1 = 0;
695             } else if (RXAdata == '!') {
696                 COMA_state = 20; // into the read mode
697                 received_CAM_countThreshold2 = 0;
698             } else if (RXAdata == '$') {
699                 COMA_state = 30; // into the read mode
700                 received_CAM_countAprilTag1 = 0;
701             } else if (RXAdata == '&') {
702                 COMA_state = 40; //EC: into the read mode
703                 received_CAM_countThreshold3 = 0;
704             }
705             else {
706                 COMA_state = 0;
707             }
708         } else if (COMA_state == 10) {
709
710             if ((received_CAM_countThreshold1 % 2) == 0) {
711                 COMALSB = RXAdata;
712             } else {
713                 DataFromCameraThreshold1.rawData[received_CAM_countThreshold1/2] = (RXAdata << 8) | COMALSB;
714             }
715             received_CAM_countThreshold1++;
716             if (received_CAM_countThreshold1 >= 4*CAMNUM_FROM_FLOATS) {
717                 received_CAM_countThreshold1 = 0;
718                 COMA_state = 0;
719                 fromCAMvaluesThreshold1[0] = DataFromCameraThreshold1.floatData[0];
720                 fromCAMvaluesThreshold1[1] = DataFromCameraThreshold1.floatData[1];
721                 fromCAMvaluesThreshold1[2] = DataFromCameraThreshold1.floatData[2];
722                 fromCAMvaluesThreshold1[3] = DataFromCameraThreshold1.floatData[3];
723                 fromCAMvaluesThreshold1[4] = DataFromCameraThreshold1.floatData[4];
724                 fromCAMvaluesThreshold1[5] = DataFromCameraThreshold1.floatData[5];
725                 fromCAMvaluesThreshold1[6] = DataFromCameraThreshold1.floatData[6];

```



```

726         fromCAMvaluesThreshold1[7] = DataFromCameraThreshold1.floatData[7];
727         fromCAMvaluesThreshold1[8] = DataFromCameraThreshold1.floatData[8];
728         NewCAMDataThreshold1 = 1; // Flag new data
729     }
730 } else if (COMA_state == 20) {
731     if ((received_CAM_countThreshold2 % 2) == 0) {
732         COMALSB = RXAdata;
733     } else {
734         DataFromCameraThreshold2.rawData[received_CAM_countThreshold2/2] = (RXAdata << 8) | COMALSB;
735     }
736     received_CAM_countThreshold2++;
737     if (received_CAM_countThreshold2 >= 4*CAMNUM_FROM_FLOATS) {
738         received_CAM_countThreshold2 = 0;
739         COMA_state = 0;
740         fromCAMvaluesThreshold2[0] = DataFromCameraThreshold2.floatData[0];
741         fromCAMvaluesThreshold2[1] = DataFromCameraThreshold2.floatData[1];
742         fromCAMvaluesThreshold2[2] = DataFromCameraThreshold2.floatData[2];
743         fromCAMvaluesThreshold2[3] = DataFromCameraThreshold2.floatData[3];
744         fromCAMvaluesThreshold2[4] = DataFromCameraThreshold2.floatData[4];
745         fromCAMvaluesThreshold2[5] = DataFromCameraThreshold2.floatData[5];
746         fromCAMvaluesThreshold2[6] = DataFromCameraThreshold2.floatData[6];
747         fromCAMvaluesThreshold2[7] = DataFromCameraThreshold2.floatData[7];
748         fromCAMvaluesThreshold2[8] = DataFromCameraThreshold2.floatData[8];
749         NewCAMDataThreshold2 = 1; // Flag new data
750     }
751 } else if (COMA_state == 30) {
752     if ((received_CAM_countAprilTag1 % 2) == 0) {
753         COMALSB = RXAdata;
754     } else {
755         DataFromCameraAprilTag1.rawData[received_CAM_countAprilTag1/2] = (RXAdata << 8) | COMALSB;
756     }
757     received_CAM_countAprilTag1++;
758     if (received_CAM_countAprilTag1 >= 4*CAMNUM_FROM_FLOATS) {
759         received_CAM_countAprilTag1 = 0;
760         COMA_state = 0;
761         fromCAMvaluesAprilTag1[0] = DataFromCameraAprilTag1.floatData[0];
762         fromCAMvaluesAprilTag1[1] = DataFromCameraAprilTag1.floatData[1];
763         fromCAMvaluesAprilTag1[2] = DataFromCameraAprilTag1.floatData[2];
764         fromCAMvaluesAprilTag1[3] = DataFromCameraAprilTag1.floatData[3];
765         fromCAMvaluesAprilTag1[4] = DataFromCameraAprilTag1.floatData[4];
766         fromCAMvaluesAprilTag1[5] = DataFromCameraAprilTag1.floatData[5];
767         fromCAMvaluesAprilTag1[6] = DataFromCameraAprilTag1.floatData[6];
768         fromCAMvaluesAprilTag1[7] = DataFromCameraAprilTag1.floatData[7];
769         fromCAMvaluesAprilTag1[8] = DataFromCameraAprilTag1.floatData[8];
770         NewCAMDataAprilTag1 = 1; // Flag new data
771     }
772 } else if (COMA_state == 40) {
773     if ((received_CAM_countThreshold3 % 2) == 0) {
774         COMALSB = RXAdata;
775     } else {
776         DataFromCameraThreshold3.rawData[received_CAM_countThreshold3/2] = (RXAdata << 8) | COMALSB;
777     }
778     received_CAM_countThreshold3++;
779     if (received_CAM_countThreshold3 >= 4*CAMNUM_FROM_FLOATS) {
780         received_CAM_countThreshold3 = 0;
781         COMA_state = 0;
782         fromCAMvaluesThreshold3[0] = DataFromCameraThreshold3.floatData[0];
783         fromCAMvaluesThreshold3[1] = DataFromCameraThreshold3.floatData[1];
784         fromCAMvaluesThreshold3[2] = DataFromCameraThreshold3.floatData[2];
785         fromCAMvaluesThreshold3[3] = DataFromCameraThreshold3.floatData[3];
786         fromCAMvaluesThreshold3[4] = DataFromCameraThreshold3.floatData[4];
787         fromCAMvaluesThreshold3[5] = DataFromCameraThreshold3.floatData[5];
788         fromCAMvaluesThreshold3[6] = DataFromCameraThreshold3.floatData[6];
789         fromCAMvaluesThreshold3[7] = DataFromCameraThreshold3.floatData[7];
790         fromCAMvaluesThreshold3[8] = DataFromCameraThreshold3.floatData[8];
791         NewCAMDataThreshold3 = 1; // Flag new data
792     }
793 }
794
795     numRXA ++;
796 }
797
798 SciaRegs.SCIFFRX.bit.RXFFINTCLR = 1;
799 PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
800
801 }
802
803 //for SerialB
804 #ifdef _FLASH
805 #pragma CODE_SECTION(RXBINT_recv_ready, ".TI.ramfunc");
806 #endif
807 __interrupt void RXBINT_recv_ready(void)
808 {
809     RXBdata = ScibRegs.SCIRXBUF.all;
810
811     /* SCI PE or FE error */
812     if (RXBdata & 0xC000) {
813         ScibRegs.SCICTL1.bit.SWRESET = 0;
814         ScibRegs.SCICTL1.bit.SWRESET = 1;
815         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
816         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;

```



```

817     } else {
818         RXBdata = RXBdata & 0x00FF;
819         // Do something with recieved character
820         numRXB ++;
821     }
822
823     ScicRegs.SCIFFRX.bit.RXFFINTCLR = 1;
824     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
825 }
826
827
828 // for SerialC
829 #ifdef _FLASH
830 #pragma CODE_SECTION(RXCINT_recv_ready, ".TI.ramfunc");
831 #endif
832 __interrupt void RXCINT_recv_ready(void)
833 {
834     RXCdata = ScicRegs.SCIRXBUF.all;
835
836     /* SCI PE or FE error */
837     if (RXCdata & 0xC000) {
838         ScicRegs.SCICTL1.bit.SWRESET = 0;
839         ScicRegs.SCICTL1.bit.SWRESET = 1;
840         ScicRegs.SCIFFRX.bit.RXFIFORESET = 0;
841         ScicRegs.SCIFFRX.bit.RXFIFORESET = 1;
842     } else {
843         RXCdata = RXCdata & 0x00FF;
844         if (read_ladar_state == 0) { //wait for the starting char 'G'
845             buf_index = 0;
846             if (RXCdata == 'G') {
847                 read_ladar_state = 1;
848                 G_count = 1;
849             }
850         } else if (read_ladar_state == 1) { //get the headers
851             G_count += 1; //count the number of headers
852             //check the status code and if it is not '0', wait for the next packet
853             if (G_count == 11) {
854                 if (RXCdata != '0') {
855                     read_ladar_state = 0;
856                 }
857             }
858             if (G_count >= 12) { //there are total 12 things before the data
859                 read_ladar_state = 2;
860                 get_dis_first = 1;
861                 dis_index = 0;
862             }
863         } else if (read_ladar_state == 2) { //get actual data
864             if (RXCdata == 0x0a) { //check for line feed
865                 LF_count += 1;
866             } else {
867                 LF_count = 0;
868                 if (get_dis_first == 1) {
869                     tempscid = (RXCdata - 0x30) << 6; //first data is the top 6 bit of the distance reading
870                     get_dis_first = 0;
871                 } else if (get_dis_first == 0) {
872                     distance = tempscid + (RXCdata - 0x30); //second data is the bottom 6 bit of the distance reading
873                     if (distance == 0 || distance == 6) { //error code
874                         distance = LADAR_MAX_READING; //set to the max reading
875                     } else if (distance == 16) { //distance is 4096mm
876                         distance = 4096;
877                     }
878                     if (LADARpingpong == 0) {
879                         //ladar_data[dis_index%228].distance_ping = distance*0.00328084; // fill in the distance_ping array
880                         checkdist = distance*0.00328084;
881                         if ((checkdist > 0.25) && (checkdist < 20.0)) {
882                             ladar_data[dis_index].distance_ping = checkdist; // fill in the distance_ping array
883                             if (dis_index == 161) {
884                                 ladar161distance = checkdist;
885                             }
886                             if (dis_index == 199) {
887                                 ladar199distance = checkdist;
888                             }
889                         } else {
890                             numRXErrors++;
891                             ladar_data[dis_index].distance_ping = ladar_data[dis_index].distance_pong;
892                             if (dis_index == 161) {
893                                 ladar161distance = ladar_data[dis_index].distance_pong;
894                             }
895                             if (dis_index == 199) {
896                                 ladar199distance = ladar_data[dis_index].distance_pong;
897                             }
898                         }
899                     }
900                 } else if (LADARpingpong == 1) {
901                     //ladar_data[dis_index%228].distance_pong = distance*0.00328084; // fill in the distance_pong array
902                     checkdist = distance*0.00328084;
903                     if ((checkdist >= 0.25) && (checkdist < 20.0)) {
904                         ladar_data[dis_index].distance_pong = checkdist; // fill in the distance_ping array
905                         if (dis_index == 161) {
906                             ladar161distance = checkdist;
907                         }
908                     }
909                 }
910             }
911         }
912     }
913 }

```

```

908         if (dis_index == 199) {
909             ladar199distance = checkdist;
910         }
911     } else {
912         numRCXerrors++;
913         ladar_data[dis_index].distance_ping = ladar_data[dis_index].distance_ping;
914         if (dis_index == 161) {
915             ladar161distance = ladar_data[dis_index].distance_ping;
916         }
917         if (dis_index == 199) {
918             ladar199distance = ladar_data[dis_index].distance_ping;
919         }
920     }
921 }
922 dis_index += 1;
923 get_dis_first = 1;
924 }
925 }
926 if (LF_count == 2) { //two consecutive line feed means the end of the current data stream
927     read_ladar_state = 0;
928     print_dis_index = dis_index;
929     PostSWI2(); // Manually cause the interrupt for the SWI2
930     LADARpingpong = 1-LADARpingpong;
931 }
932 }
933 buf[buf_index % 477] = RXCdata;//fill in the buf array, debug purpose
934 buf_index++;
935 numRXC ++;
936 }
937 }
938
939 ScicRegs.SCIFFRX.bit.RXFFINTCLR = 1;
940 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
941 }
942
943
944 //for SerialD
945 #ifdef _FLASH
946 #pragma CODE_SECTION(RXDINT_recv_ready, ".TI.ramfunc");
947 #endif
948 __interrupt void RXDINT_recv_ready(void)
949 {
950     uint16 t i = 0;//for loop
951     RXDdata = ScidRegs.SCIRXBUF.all;
952
953     /* SCI PE or FE error */
954     if (RXDdata & 0xC000) {
955         ScidRegs.SCICTL1.bit.SWRESET = 0;
956         ScidRegs.SCICTL1.bit.SWRESET = 1;
957         ScidRegs.SCIFFRX.bit.RXFIFORESET = 0;
958         ScidRegs.SCIFFRX.bit.RXFIFORESET = 1;
959     } else {
960         RXDdata = RXDdata & 0x00FF;
961         if (com_state == 0) {
962             if (RXDdata == '*') {
963                 com_state = 1; //for receiving path
964             } else if (RXDdata == '0') {
965                 com_state = 2; //for receiving optitrack data
966             } else {
967                 com_state = 0;
968             }
969         } else if (com_state == 1) {
970             if (RXDdata == '*') {
971                 com_state = 10; // into the read mode
972                 received_path_count = 0;
973                 newAstarPath = 0;
974             } else if (RXDdata == '$') {
975                 com_state = 30;
976                 received_LV_count = 0;
977             } else if (RXDdata == '!') {
978                 com_state = 40;
979                 received_CMD_count = 0;
980             } else {
981                 com_state = 0;
982             }
983         } else if (com_state == 10) { // this should not occurred left it here for future code with A* path planning
984             path_received[received_path_count] = RXDdata;
985             received_path_count++;
986             if (received_path_count >= 81) {
987                 received_path_count = 0;
988                 com_state = 0;
989                 newAstarPath = 1;
990             }
991         } else if (com_state == 2) {
992             if (RXDdata == 'p'){
993                 com_state = 20;
994             } else {
995                 com_state = 0;
996             }
997         } else if (com_state == 20) {
998             if (RXDdata == 't'){

```

```

999         com_state = 21;
1000     } else {
1001         com_state = 0;
1002     }
1003 } else if (com_state == 21) {
1004     if (RXDdata == 'i'){
1005         com_state = 22;
1006     } else {
1007         com_state = 0;
1008     }
1009 } else if (com_state == 22) {
1010     if (RXDdata == '.'){
1011         com_state = 23;
1012         // received_opti_count = 0;
1013     } else {
1014         com_state = 0;
1015     }
1016 } else if (com_state == 23) {
1017     if (received_opti_count % 2 == 0){ // this is for motion tracking system left for future code
1018         tempLSB = RXDdata;
1019     } else {
1020         optirx.rawData[received_opti_count/2] = (RXDdata << 8)|tempLSB;
1021     }
1022     received_opti_count++;
1023     if (received_opti_count >= OPTITRACKDATASIZE*4) {
1024         if (new_optitrack == 0) {
1025             for (i=0;i<OPTITRACKDATASIZE;i++) { // x, y, heading, trackableID, framecount
1026                 Optitrackdata[i] = optirx.floatData[i];
1027             }
1028             new_optitrack = 1;
1029         }
1030         received_opti_count = 0;
1031         com_state = 0;
1032     }
1033 } else if (com_state == 30) {
1034     if ((received_LV_count % 2) == 0) {
1035         tempLSB = RXDdata;
1036     } else {
1037         DataFromLabView.rawData[received_LV_count/2] = (RXDdata << 8)|tempLSB;
1038     }
1039     received_LV_count++;
1040     if (received_LV_count >= 4*LVNUM_TOFROM_FLOATS) {
1041         received_LV_count = 0;
1042         com_state = 0;
1043         fromLVvalues[0] = DataFromLabView.floatData[0];
1044         fromLVvalues[1] = DataFromLabView.floatData[1];
1045         fromLVvalues[2] = DataFromLabView.floatData[2];
1046         fromLVvalues[3] = DataFromLabView.floatData[3];
1047         fromLVvalues[4] = DataFromLabView.floatData[4];
1048         fromLVvalues[5] = DataFromLabView.floatData[5];
1049         fromLVvalues[6] = DataFromLabView.floatData[6];
1050         fromLVvalues[7] = DataFromLabView.floatData[7];
1051         NewLVData = 1; // Flag new data
1052     }
1053 } else if (com_state == 40) {
1054     if ((received_CMD_count % 2) == 0) {
1055         tempLSB = RXDdata;
1056     } else {
1057         DataFromLinuxCMD.rawData[received_CMD_count/2] = (RXDdata << 8)|tempLSB;
1058     }
1059     received_CMD_count++;
1060     if (received_CMD_count >= 4*CMDNUM_FROM_FLOATS) {
1061         received_CMD_count = 0;
1062         com_state = 0;
1063
1064         LinuxCommands[0] = DataFromLinuxCMD.floatData[0];
1065         LinuxCommands[1] = DataFromLinuxCMD.floatData[1];
1066         LinuxCommands[2] = DataFromLinuxCMD.floatData[2];
1067         LinuxCommands[3] = DataFromLinuxCMD.floatData[3];
1068         LinuxCommands[4] = DataFromLinuxCMD.floatData[4];
1069         LinuxCommands[5] = DataFromLinuxCMD.floatData[5];
1070         LinuxCommands[6] = DataFromLinuxCMD.floatData[6];
1071         LinuxCommands[7] = DataFromLinuxCMD.floatData[7];
1072         LinuxCommands[8] = DataFromLinuxCMD.floatData[8];
1073         LinuxCommands[9] = DataFromLinuxCMD.floatData[9];
1074         LinuxCommands[10] = DataFromLinuxCMD.floatData[10];
1075         newLinuxCommands = 1; // Flag new data
1076     }
1077 }
1078 }
1079 }
1080 numRXD ++;
1081 ScidRegs.SCIFFRX.bit.RXFFINTCLR = 1;
1082 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
1083 }
1084
1085
1086 // SerialA only setup for Tera Term connection
1087 char serial_printf_bufSCIA[BUF_SIZE_SCIA];
1088
1089 uint16_t serial_printf(serialSCIA_t *s, char *fmt, ...)

```

```

1090 {
1091     va_list ap;
1092
1093     va_start(ap,fmt);
1094     vsprintf(serial_printf_bufSCIA,fmt,ap);
1095     va_end(ap);
1096
1097     return serial_sendSCIA(s,serial_printf_bufSCIA,strlen(serial_printf_bufSCIA));
1098 }
1099
1100 //For Text LCD SCIB only
1101 char UART_printf_bufferSCIB[BUF_SIZEESCIB];
1102 void UART_vprintfLine(unsigned char line, char *format, va_list ap)
1103 {
1104     char sendmsg[24];
1105
1106     int i;
1107
1108     vsprintf(UART_printf_bufferSCIB,format,ap);
1109
1110     // Add header information and pad end of transfer with spaces to clear display
1111     sendmsg[0] = 0xFE;
1112     sendmsg[1] = 'G';
1113     sendmsg[2] = 1;
1114     sendmsg[3] = line;
1115     for (i=4;i<24;i++) {
1116         if (i >= strlen(UART_printf_bufferSCIB)+4) {
1117             sendmsg[i] = ' ';
1118         } else {
1119             sendmsg[i] = UART_printf_bufferSCIB[i-4];
1120         }
1121     }
1122     serial_sendSCIB(&SerialB,sendmsg,24);
1123 }
1124
1125 void UART_printfLine(unsigned char line, char *format, ...)
1126 {
1127     va_list ap;
1128     va_start(ap, format);
1129     UART_vprintfLine(line,format,ap);
1130 }

```